NOMBRE DEL PROYECTO (BIKENEXUS)

DOMINIO: LOGÍSTICA

ESTUDIANTES:
ATHINA ALEJANDRA CAPPELLETTI GARCÍA, CODIGO: 1000327379

DAVID ALEJANDRO GUTIÉRREZ LEAL, CODIGO: 1000345861

MARIAMNY DEL VALLE RAMIREZ TELLES, CODIGO: 1000359603

GINNA ALEJANDRA VALENCIA MACUACE, CODIGO: 1000358197

EMMANUEL ALVAREZ CASTRILLÓN, CODIGO: 1000357782

UNIVERSIDAD EAFIT
ESCUELA DE CIENCIAS E INGENIERÍA
ASIGNATURA: SISTEMAS INTENSIVOS DE DATOS

DOCENTE:
ANDRÉS SACRE ALZATE

MEDELLÍN, COLOMBIA
30 DE SEPTIEMBRE DE 2026

Definición del problema

Los sistemas de bicicletas compartidas ofrecen una alternativa de movilidad urbana que depende de un equilibrio operativo constante: los usuarios deben encontrar bicicletas disponibles cuando quieren iniciar un viaje y espacios libres cuando necesitan finalizarlo. Cuando una estación se queda sin bicicletas, se pierde una oportunidad de viaje; cuando se llena por completo, los usuarios no pueden devolver la bicicleta y deben buscar otra estación. Ambos casos reducen la calidad del servicio, generan frustración, aumentan tiempos de desplazamiento y pueden disminuir la confianza de los usuarios en el sistema.

El problema de negocio consiste en que los operadores de una red de bicicletas compartidas necesitan identificar y priorizar, con suficiente anticipación, las estaciones con riesgo de quedarse sin bicicletas o sin espacios disponibles, para tomar decisiones de reabastecimiento y redistribución más oportunas. La operación no puede depender únicamente de revisiones manuales o de promedios históricos: la disponibilidad cambia continuamente según la hora, el día de la semana, las condiciones climáticas, los eventos en la ciudad, la ubicación de la estación y los patrones de viaje entre zonas residenciales, comerciales, educativas y de transporte.

Por ejemplo, una estación cercana a una terminal de transporte público puede recibir muchas bicicletas durante la mañana, cuando las personas llegan a trabajar, y perderlas rápidamente al final de la tarde, cuando regresan a sus hogares. De forma similar, las estaciones ubicadas en zonas turísticas o recreativas pueden experimentar picos de demanda durante fines de semana, días festivos o condiciones climáticas favorables. Si el equipo operativo conoce estas situaciones solo después de que una estación se vacía o se llena, la respuesta llega tarde y el impacto sobre los usuarios ya ocurrió.

El sistema busca transformar los datos operativos e históricos disponibles en información accionable para que la empresa pueda mantener un nivel adecuado de servicio. No se trata únicamente de saber cuántas bicicletas hay en cada estación, sino de responder a las preguntas posteriormente mencionadas.

Preguntas analíticas y operacionales

El sistema debe responder qué estaciones están vacías, llenas o en riesgo de llegar a una condición crítica en los próximos minutos; cuáles deben ser priorizadas por los equipos de redistribución; y en qué zonas se concentran los desequilibrios entre bicicletas disponibles y demanda. También debe permitir analizar patrones históricos de viajes por estación, hora, día, temporada y condiciones externas, para identificar zonas con demanda recurrente, estimar viajes potencialmente perdidos y decidir dónde aumentar capacidad o instalar nuevas estaciones.

Usuarios del sistema y decisiones que toman

Los operadores de redistribución usarán el sistema para decidir qué estaciones atender primero, cuántas bicicletas trasladar y cómo priorizar sus recorridos. Los supervisores de operaciones lo utilizarán para asignar vehículos, personal y recursos durante horas de alta demanda o eventos excepcionales. Por su parte, los analistas de negocio y planeación analizarán tendencias históricas para recomendar cambios de capacidad, reubicación de bicicletas, expansión de estaciones o ajustes en la operación, mientras que la gerencia evaluará indicadores de calidad del servicio, costos operativos y satisfacción de los usuarios.

Justificación para una arquitectura de datos distribuida

Este problema justifica una arquitectura de datos distribuida porque combina alto volumen histórico, generación continua de nuevos datos y variedad de formatos y estructuras. Una red de bicicletas compartidas puede generar millones de registros de viajes a lo largo de varios años. Cada viaje contiene información como estación de origen, estación de destino, hora de inicio, hora de finalización, duración y atributos del usuario. Analizar varios años de estos registros requiere procesar grandes volúmenes para identificar patrones por estación, zona, hora y temporada.

Además del historial de viajes, el sistema debe incorporar actualizaciones frecuentes sobre el estado operativo de las estaciones: bicicletas disponibles, espacios libres, estaciones fuera de servicio y otros cambios que se producen durante el día. Estos datos tienen naturaleza temporal y cambian continuamente; por ello, su valor disminuye si se procesan con demasiada demora. La disponibilidad de una estación hace treinta minutos puede no representar su situación actual ni ser suficiente para tomar decisiones operativas.

La variedad también es importante. El proyecto integra datos históricos estructurados de viajes, datos semiestructurados en formato JSON provenientes del estado de estaciones y, potencialmente, datos externos como clima, calendario, festivos o eventos urbanos. Preparar, limpiar, relacionar y analizar estas fuentes exige manejar diferencias de formato, frecuencia, calidad y granularidad temporal.

Una base de datos relacional tradicional puede ser adecuada para registrar transacciones operativas puntuales o consultar un volumen limitado de información. Sin embargo, se vuelve menos conveniente cuando se requiere conservar grandes históricos, procesar actualizaciones frecuentes, ejecutar agregaciones temporales complejas y combinar múltiples fuentes de datos con estructuras distintas. Por esta razón decidimos que una arquitectura de datos distribuidas es algo crucial para este proyecto.

En síntesis, el sistema busca mejorar la disponibilidad de bicicletas y espacios en una red de movilidad compartida. Al convertir datos de viajes y estados operativos en alertas e indicadores, permitirá que los equipos de operación actúen antes de que una estación falle para el usuario, mientras que los equipos de planeación podrán tomar decisiones fundamentadas sobre capacidad, infraestructura y distribución de recursos.

Requisitos Funcionales:

- RF-01: El sistema debe ingestar el estado actual de todas las estaciones (bicicletas disponibles, espacios libres y si la estación está activa) cada 60 segundos, y publicarlo en un topic de Kafka con una latencia menor a 5 segundos desde que se genera el dato.

- RF-02: El sistema debe cargar mensualmente los registros del histórico de viajes (estación de origen, estación de destino, hora de inicio y hora de fin) desde un bucket de S3 en formato CSV hacia la capa Bronze, sin pérdida de registros (el conteo de Bronze debe coincidir con el del archivo fuente).

- RF-03: El sistema debe cargar diariamente los datos fijos de las estaciones (identificador, nombre, ubicación, capacidad y zona) desde la fuente de información de estaciones hacia la capa Bronze, garantizando que toda estación que aparezca en el flujo de estado en tiempo real exista en este catálogo.

- RF-04: El sistema debe limpiar de forma continua (cada 30 segundos) los registros de estado de estaciones almacenados en la capa Bronze, eliminando los reportes duplicados y apartando los registros inválidos (por ejemplo, bicicletas disponibles mayores que la capacidad de la estación o estaciones con capacidad igual a 0), de modo que en la capa Silver no exista ningún registro duplicado ni con valores fuera de rango.

- RF-05: El sistema debe limpiar mensualmente los registros del histórico de viajes almacenados en la capa Bronze, apartando los viajes con duración menor a 1 minuto o mayor a 24 horas y los viajes cuya estación de origen o destino no exista en el catálogo de estaciones, de modo que en la capa Silver no exista ningún viaje con duración fuera de rango ni con estaciones inexistentes.

- RF-06: El sistema debe calcular el porcentaje de ocupación de cada estación y su tendencia de cambio usando una ventana deslizante (sliding window) de 15 minutos con avance de 5 minutos, y clasificar cada estación como vacía (0 bicicletas), llena (0 espacios libres), en riesgo (si al ritmo actual quedaría vacía o llena en menos de 15 minutos) o normal (en cualquier otro caso).

- RF-07: El sistema debe generar una alerta cuando una estación sea clasificada como vacía, llena o en riesgo, y entregar a los operadores una lista de estaciones ordenada por el tiempo estimado que les queda para quedar vacías o llenas (de menor a mayor), en menos de 5 segundos desde su clasificación.

- RF-08: El sistema debe construir y actualizar mensualmente en la capa Gold un modelo dimensional (star schema) con una tabla de hechos de viajes y dimensiones de estación, tiempo y clima, que permita a los analistas consultar el número de viajes y su duración

promedio por estación, hora, día y condición climática, con un tiempo de respuesta menor a 5 segundos.

Requisitos no funcionales

- RNF-01 (Escalabilidad): El sistema debe soportar un pico de al menos 10 veces la carga normal de eventos (carga normal = N estaciones ÷ 60 segundos entre reportes), manteniendo la latencia de extremo a extremo de las alertas por debajo de 1 minuto (RNF-04), y debe poder escalar horizontalmente aumentando el número de particiones del topic de Kafka y de executors de Spark.

- RNF-02 (Disponibilidad): Ante una partición de red, el sistema debe priorizar la Disponibilidad (A) sobre la Consistencia (C), adoptando una posición **AP** del teorema CAP con consistencia eventual: debe seguir respondiendo con el último estado conocido de cada estación, indicando la hora de su último reporte, y marcar como desactualizada cualquier estación sin reportes en más de 2 minutos. Esto es aceptable porque un atraso de segundos no cambia la decisión de redistribución del operador, a diferencia de un sistema financiero, donde un dato desactualizado sí genera errores.

- RNF-03 (Consistencia): El sistema debe garantizar entrega at-least-once en el tramo productor → Kafka → Bronze, configurando el productor con acks=all y reintentos, y usando checkpoints en Spark Structured Streaming. Los posibles duplicados deben eliminarse al pasar a Silver usando como clave de deduplicación el identificador de la estación y el timestamp del reporte, de modo que Silver refleje un efecto exactly-once: no debe existir más de un registro por estación y timestamp.

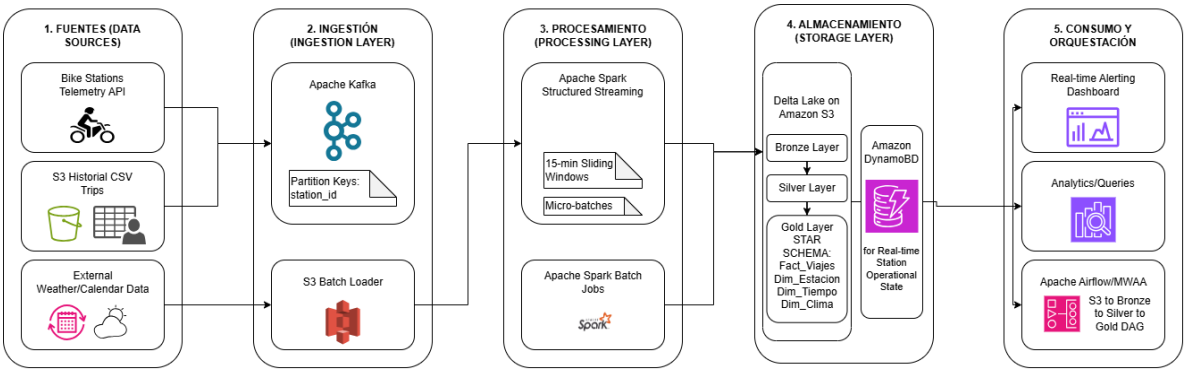
- RNF-04 (Latencia): En el camino de streaming, el tiempo desde que una estación genera su reporte hasta que la alerta correspondiente llega al operador debe ser menor a 1 minuto, usando el modo de salida update de Spark Structured Streaming para emitir resultados sin esperar el cierre de la ventana. En el camino batch, los viajes de un mes deben estar disponibles para consulta en la capa Gold dentro de las 24 horas siguientes a la publicación del archivo mensual.

- RNF-05 (Tolerancia a Fallos): Si cae un broker de Kafka, una réplica ISR debe ser elegida automáticamente como nuevo líder de sus particiones en pocos segundos, sin pérdida de datos, gracias a un factor de replicación de 3 y ***min.insync.replicas = 2***, lo que permite tolerar la caída de un broker. Ahora, si el job de Spark Structured Streaming falla, debe reanudarse desde el último offset confirmado usando el checkpoint almacenado en S3 (que guarda los offsets leídos y el estado de las ventanas), sin pérdida de datos. Los registros del micro-batch reprocesado que resulten duplicados se eliminan en Silver mediante la deduplicación definida en el RNF-03.

- RNF-06 (Trazabilidad): El sistema debe permitir auditar el linaje de cualquier dato desde Gold hasta su fuente mediante: (1) el ***_delta_log*** de Delta Lake con ***time travel***, conservando el historial de versiones de cada tabla por al menos 30 días; (2) columnas de metadatos en cada registro (topic, partición y offset de Kafka, o nombre del archivo de origen en el caso batch, más la fecha de ingesta); y (3) la capa Bronze inmutable, que conserva el dato crudo original. De este modo, para cualquier registro de Silver o Gold debe poder identificarse el mensaje o archivo exacto del que proviene.
- RNF-07 (Evolución de esquema): El sistema debe absorber cambios compatibles en el JSON de estado de estaciones (por ejemplo, un campo opcional nuevo) sin detener el pipeline ni reprocesar el histórico, y debe rechazar sin perder datos los registros incompatibles. Bronze conserva el mensaje original completo en una columna `payload` además de las columnas parseadas, de modo que un campo nuevo nunca se pierde. Delta Lake aplica *schema enforcement* en Silver y *schema evolution* (`mergeSchema`) para promover columnas nuevas en el siguiente despliegue planificado. Los registros con campos obligatorios ausentes o tipos inválidos van a una tabla de cuarentena con el motivo del rechazo. Criterio verificable: Silver + cuarentena = Bronze (0 registros perdidos).

- RNF-08 (Seguridad y privacidad): El sistema debe proteger los datos en tránsito y en reposo, y restringir el acceso por rol. Esto incluye TLS en tránsito, cifrado en reposo en S3 y DynamoDB, y permisos IAM de mínimo privilegio: el job de streaming solo escribe en Bronze, Silver y DynamoDB, y los analistas solo leen Gold. Los identificadores de usuario de los viajes deben seudonimizarse (hash con sal) al pasar a Silver, de modo que Gold no contenga identificadores directos, en línea con la Ley 1581 de 2012 (hábeas data). En el ambiente local, las credenciales se manejan por variables de entorno y usuarios separados en MinIO.

Diagrama de arquitectura



Explicación de cada parte del diagrama:

1. Fuentes de Datos (Data Sources)

Bike Stations Telemetry API: Es la fuente en tiempo real. Son los datos que envían automáticamente las N estaciones de bicicletas cada 60 segundos (vía JSON) indicando cuántas bicis disponibles tienen, cuántos espacios libres quedan y si la estación está activa.

S3 Historical CSV Trips: Es la fuente batch o histórica. Representa los archivos .csv almacenados en Amazon S3 que contienen el registro de todos los viajes pasados realizados por los usuarios (estación de origen, destino, hora de inicio y fin).

External Weather / Calendar Data: Son datos externos complementarios (clima, si el día es festivo o si hay eventos en la ciudad). Sirven para entender por qué sube o baja el uso de bicicletas según la lluvia o los días de descanso.

2. Capa de Ingestión (Ingestion Layer)

Apache Kafka: Es el "autopista de peaje" para los datos en tiempo real. Recibe de forma continua miles de mensajes de las estaciones, asegurando que no se pierdan y organizándolos en orden por estación mediante una clave (station_id).

S3 Batch Loader: Es el mecanismo encargado de tomar los archivos CSV históricos desde S3 y subirlos/cargarlos de manera ordenada una vez al día hacia la capa Bronze del Data Lake.

3. Capa de Procesamiento (Processing Layer)

Apache Spark Structured Streaming: Es el motor de procesamiento rápido. Examina los datos en tiempo real cada pocos segundos, calcula el porcentaje de ocupación usando ventanas de 15 minutos y detecta inmediatamente qué estaciones están en riesgo de quedar vacías o llenas.

Apache Spark Batch Jobs: Es el mismo motor Spark, pero usado para tareas pesadas que se ejecutan **cada mes, cuando se publica un nuevo archivo de viajes**. Procesa el volumen masivo de viajes **del mes**, limpia errores (ej. viajes de menos de 1 minuto **o con estaciones inexistentes**) y cruza la información con el clima y las estaciones **para construir el star schema de la capa Gold**.

4. Capa de Almacenamiento (Storage Layer)

Delta Lake on Amazon S3 (Medallion Architecture): Es el lago de datos organizado en tres niveles de calidad sobre S3:

Bronze Layer: Guarda los datos tal cual llegan (crudos e inmutables) para mantener la trazabilidad de la fuente original.

Silver Layer: Almacena los datos limpios, filtrados y sin duplicados.

Gold Layer: Organiza la información en un modelo analítico en estrella (Star Schema: Fact_Viajes, Dim_Estacion, Dim_Tiempo, Dim_Clima) optimizado para reportes rápidos.

Amazon DynamoDB: Es una base de datos ultra rápida no relacional (NoSQL). Guarda únicamente el último estado conocido de cada estación para que el sistema operacional consulte el estado actual sin tener que escanear todo el Data Lake.

5. Capa de Consumo y Orquestación

Real-time Alerting Dashboard: Es el panel visual que usan los operadores de redistribución. Les muestra la lista de estaciones priorizadas que se van a quedar vacías o llenas en menos de 15 minutos para que envíen camiones a mover bicicletas.

Analytics / Queries: Son las herramientas de consulta (como AWS Athena o Power BI) que usan los analistas de negocio para analizar tendencias históricas de viajes, evaluar demanda por zonas y decidir dónde construir nuevas estaciones.

Apache Airflow / MWAA: Es el "director de orquesta" del sistema. Se encarga de programar, ejecutar y monitorear el flujo de trabajo diario de los datos (S3 → Bronze → Silver → Validación → Gold) asegurando que cada paso se ejecute en el orden correcto.

Justificación de decisiones técnicas

Las decisiones técnicas se estructuran mediante ADR (Architecture Decision Record), considerando las necesidades de procesamiento en tiempo real, procesamiento batch, almacenamiento y consumo de los datos. Se priorizan servicios de AWS para reducir la complejidad de infraestructura y facilitar la escalabilidad del sistema.

ADR-01: Selección del mecanismo de streaming y garantía de entrega

1. Contexto

El sistema debe recibir periódicamente el estado de aproximadamente N estaciones de bicicletas. Estos eventos deben procesarse continuamente para identificar estaciones con riesgo de quedar vacías o saturadas y generar alertas con una latencia inferior a un minuto.

Además, la solución debe tolerar fallos y evitar la pérdida de eventos durante la transmisión.

2. Alternativas consideradas

- **Amazon Kinesis Data Streams:** servicio administrado de AWS para la ingesta de eventos en tiempo real, con escalabilidad mediante particiones.
- **Apache Kafka:** solución ampliamente utilizada para streaming distribuido y relectura de eventos, pero requiere mayor administración de infraestructura si se despliega directamente.
- **RabbitMQ:** adecuado para mensajería, pero menos orientado al almacenamiento y reprocesamiento de grandes volúmenes de eventos.

3. Decisión

Se selecciona **Apache Kafka** como mecanismo de ingesta de eventos en tiempo real. Se configura una retención de datos de al menos 24 horas y se utiliza `estacion_id` como clave del mensaje (*message key*), de manera que los reportes de una misma estación se asignen a la misma partición y se mantenga su orden dentro de ella.

Se adopta una garantía **At-Least-Once**. Ante fallos de comunicación, los reintentos del productor pueden generar eventos duplicados, por lo que la deduplicación se realizará durante el procesamiento con Spark.

4. Consecuencias

La principal ventaja es disponer de una solución distribuida, escalable y con control sobre la configuración de particiones, replicación y retención de eventos. Además, permite integrar la ingesta con Spark Structured Streaming y facilita el reprocesamiento de datos. Como desventaja, el equipo debe administrar y monitorear los brokers de Kafka, configurar adecuadamente la

replicación y gestionar los recursos de infraestructura. También es necesario controlar los posibles duplicados durante el procesamiento.

ADR-02: Selección del motor de procesamiento

1. Contexto

El sistema debe calcular continuamente el porcentaje de ocupación, las tendencias y el riesgo de desabastecimiento o saturación. Para ello se requieren ventanas deslizantes de 15 minutos con avances de 5 minutos.

2. Alternativas consideradas

- **Spark Structured Streaming:** permite procesar eventos continuamente y utilizar las mismas herramientas de Spark para los procesos batch.
- **Apache Flink:** ofrece procesamiento de eventos con muy baja latencia y manejo avanzado de estado, pero introduce un motor adicional en la arquitectura.
- **Procesamiento batch:** simplifica la solución, pero no permite responder oportunamente a los cambios del estado de las estaciones.

3. Decisión

Se selecciona **Apache Spark con Structured Streaming** utilizando micro-batches. Spark permite realizar las transformaciones necesarias sobre ventanas temporales y reutilizar el mismo motor para procesar diariamente el histórico de viajes.

Se utilizarán checkpoints para permitir la recuperación del procesamiento ante fallos y mecanismos de watermark para controlar eventos que lleguen con retraso.

4. Consecuencias

La solución no alcanza las latencias de milisegundos que podría ofrecer Flink, pero cumple el requisito operativo del proyecto y evita incorporar un segundo motor de procesamiento.

ADR-03: Selección del almacenamiento y modelo de datos de Gold

1. Contexto

El sistema debe conservar los datos históricos y procesados en una arquitectura Bronze/Silver/Gold. La capa Gold debe permitir consultas analíticas sobre viajes, estaciones, tiempo y condiciones climáticas.

2. Alternativas consideradas

- **Parquet sobre Amazon S3:** ofrece almacenamiento eficiente, pero no proporciona por sí mismo capacidades como transacciones ACID y control de versiones.
- **Modelo relacional normalizado:** facilita la integridad de los datos, pero puede aumentar la cantidad de joins necesarios para las consultas analíticas.

- **Delta Lake:** añade capacidades de gestión transaccional sobre almacenamiento en objetos y permite trabajar con procesos batch y streaming.

3. Decisión

Se selecciona **Delta Lake sobre Amazon S3**, organizado en las capas Bronze, Silver y Gold.

La capa Gold utilizará un **modelo dimensional en estrella**, compuesto principalmente por:

- Fact_Viajes
- Dim_Estacion
- Dim_Tiempo
- Dim_Clima

Delta Lake proporciona transacciones ACID y mantiene el registro _delta_log, permitiendo consultar versiones anteriores mediante **Time Travel** y conservar la trazabilidad de los datos durante el período definido por el proyecto.

4. Consecuencias

Se obtiene un almacenamiento adecuado para procesos analíticos y se facilita la integración entre datos batch y streaming. Como desventaja, el modelo dimensional puede generar cierta redundancia y requiere tareas de mantenimiento del almacenamiento.

ADR-04: Componente adicional para el estado operacional

1. Contexto

Aunque Delta Lake permite almacenar los datos procesados, la aplicación operacional necesita consultar rápidamente el último estado conocido de cada estación para mostrar disponibilidad y generar alertas. Consultar directamente el Data Lake para cada actualización del panel no es apropiado para este tipo de acceso.

2. Alternativas consideradas

- **Delta Lake:** adecuado para procesamiento y análisis, pero no está optimizado para consultas operacionales frecuentes por clave.
- **Amazon ElastiCache/Redis:** ofrece lecturas de muy baja latencia, pero requiere administrar una capa de caché adicional.
- **Amazon DynamoDB:** proporciona acceso por clave con baja latencia y es un servicio administrado de AWS.

3. Decisión

Se selecciona **Amazon DynamoDB** como capa de consulta operacional. Cada estación tendrá estacion_id como clave primaria y almacenará su último estado procesado.

El job de **Spark Structured Streaming** actualizará continuamente el estado de cada estación en DynamoDB. La escritura será idempotente para evitar inconsistencias ante posibles reprocesamientos.

La solución adopta una postura **AP (Availability y Partition Tolerance)** del teorema CAP, priorizando que los operadores puedan consultar el sistema incluso ante problemas temporales de comunicación.

4. Consecuencias

El dashboard puede consultar rápidamente el estado de las estaciones sin acceder directamente al Data Lake. La principal consecuencia es aceptar **consistencia eventual**, por lo que durante una falla podría mostrarse temporalmente el último estado conocido.

ADR-05: Herramienta de orquestación

1. Contexto

El procesamiento diario requiere coordinar varias tareas: cargar los archivos históricos desde Amazon S3, procesarlos en Bronze y Silver, validar los datos y construir las tablas de Gold. Estas tareas tienen dependencias que deben ejecutarse en un orden determinado.

2. Alternativas consideradas

- **Cron y scripts:** solución sencilla, pero limitada para gestionar dependencias, reintentos y monitoreo.
- **AWS Step Functions:** permite coordinar servicios de AWS, pero orienta la solución hacia flujos específicos del ecosistema AWS.
- **Apache Airflow:** permite representar las dependencias mediante DAG y administrar ejecuciones, reintentos y monitoreo.

3. Decisión

Se selecciona **Apache Airflow** como herramienta de orquestación. El DAG diario seguirá, de forma general, el flujo:

S3 → Bronze → Silver → validación de calidad → Gold

Airflow se encargará de coordinar las tareas, mientras que Spark realizará las transformaciones.

4. Consecuencias

Se obtiene un flujo organizado y observable, con posibilidad de controlar dependencias y reintentos. Como contrapartida, Airflow agrega un componente adicional que debe ser administrado, aunque puede utilizarse mediante un servicio administrado como Amazon MWAA.

Plan de trabajo

Distribución de responsabilidades:

Integrante	Componente	Requisitos	Revisor
Mariamny	Ingesta en streaming: productor simulado, broker y Spark Streaming → Bronze y Silver (con deduplicación)	RF-01, RF-04, RNF-03	Emmanuel
Ginna	Analítica en streaming: ventanas deslizantes, clasificación de estaciones, alertas y estado en DynamoDB	RF-06, RF-07, RNF-02	David
David	Cargas batch (viajes, estaciones, clima y calendario) y Silver batch, con cuarentena de registros inválidos y evolución de esquema	RF-02, RF-03, RF-05, RNF-07	Mariamny
Emmanuel	Infraestructura y orquestación: Docker Compose, DAG de Airflow, pruebas de carga y de fallos, seguridad	RNF-01, RNF-05, RNF-08	Athina
Athina	Capa Gold (star schema), trazabilidad con Delta Lake, medición de latencia batch y dashboards de consumo	RF-08, RNF-04, RNF-06	Ginna

Cronograma

Hito	Fecha	Qué debe estar listo	Requisitos cubiertos
H0	Viernes 9 de ocutbre	Decisiones congeladas (broker, ambiente, nombres), contratos de datos (esquemas JSON/Delta), repo con carpetas y PR abierto	-
H1: Base	Viernes 16 de octubre	`docker-compose` up levanta broker, Spark, MinIO y Airflow. Un evento viaja del productor a Bronze (con metadatos de origen), el CSV de viajes y el catálogo de estaciones llegan a Bronze, y existe un DAG con las tareas conectadas	RF-01, RF-02, RF-03
Integración	Miércoles 21 de octubre	Primera corrida de extremo a extremo con datos reales: Silver streaming y batch (deduplicación y reglas de calidad) y un Gold mínimo consultable	RF-04, RF-05
H2: funcionalidad completa	Viernes 23 de octubre	Ventanas deslizantes, clasificación y alertas con estado en DynamoDB; cuarentena y evolución de esquema; star schema completo y DAG completo; trazabilidad con `_delta_log`, time travel y metadatos	RF-06, RF-07, RF-08, RNF-02, RNF-03, RNF-06, RNF-07

Pruebas	Martes 27 de octubre	Carga de 330 eventos por segundo, caída de broker, reinicio de Spark desde el checkpoint, medición de latencias (alerta y batch antes de las 6:00 a. m.) y revisión de seguridad (cifrado y permisos)	RNF-01, RNF-04, RNF-05, RNF-08
Correcciones	Jueves 29 de octubre	Corrección de errores. Primer ensayo completo del demo.	-
H3: Entrega	Sábado 31 de octubre	README, bitácora IA, y diapositivas. PR final, segundo ensayo del demo y revisión de que cada integrante pueda responder por cualquier parte del sistema.	

Riesgos y mitigación

Riesgo	**Mitigación**	**Verificación**
No hay una fuente real de streaming	Productor que reproduce el histórico de viajes y emite el estado de las estaciones cada 30 segundos (configurable). Se documenta como simulador de eventos, no como carga batch.	H1, 16 de octubre
Integración tardía entre componentes	Contratos de datos definidos en H0, datos de ejemplo para que nadie espere a otro, integración intermedia el 21 de octubre y un esqueleto de extremo a extremo desde H1.	Integración, 21 de ocutbre
Falla durante la demo en vivo	Ensayos completos el 29 y el 30 de octubre, dataset precargado, script de reinicio y saber explicar la causa de cualquier fallo.	Correcciones, 29 de cotubre
El entorno (Kafka, Spark, MinIO y Airflow en Docker) consume más memoria de la que tienen los portátiles	Un solo worker de Spark, Airflow con LocalExecutor, límites de memoria por contenedor, número de estaciones parametrizable y una máquina de referencia para el demo (ver explicación abajo)	H1, 16 de octubre